What the RGAT Actually Does

September 7, 2026

The setting. Every node is one visit, and it starts life as a vector — so far, the MiniLM embedding of its text. Nodes are joined by two different kinds of edge: one joins visits whose text is similar (the kNN edges), the other joins visits that belong to the same patient. A colour is a patient, and the prediction we care about is made per colour.

One layer, in one sentence. A layer rewrites each node's vector as a weighted average of its neighbours' vectors, plus a copy of the node itself. Stack three layers and a node has absorbed information from everything within three steps of it in the graph.

The "AT": the weights are learned. A plain graph network treats all neighbours alike. Attention does not. For every edge it computes a score, and the scores of one node's neighbours are turned into weights that add up to one:

$$e_{ij} \;=\; \mathrm{LeakyReLU}\Big(a^{\top}\,[\,Wh_i \,\|\, Wh_j\,]\Big), \qquad \alpha_{ij} \;=\; \frac{\exp(e_{ij})}{\sum_{j'\in\mathcal{N}(i)}\exp(e_{ij'})}.$$

Here h_i is the node's current vector, W is a matrix the model learns, and a is a vector the model learns. Read α_{ij} as "how much node i should listen to neighbour j", decided from the pair itself rather than fixed in advance. The whole thing is done four times in parallel with four separate copies of W and a — the *heads* — so the model can attend to four different things at once, and the four results are laid side by side.

The "R": each kind of edge is treated separately. This is the part that matters for our graph. Each relation gets its own W and its own a. The weights are normalised *within* a relation, so a node's text-similar neighbours compete for attention only against each other, and its same-patient neighbours only against each other. Each relation then produces its own full weighted average, and the layer adds them up:

$$h_i' \;=\; \sum_{r\in\text{relations}} \;\sum_{j\in\mathcal{N}_r(i)} \alpha_{ij}^{r}\, W_r h_j \;+\; W_{\text{self}}\, h_i.$$

So a node's new vector is: what its similar visits say, plus what its own patient's other visits say, plus itself. Because the two are added rather than averaged together, each relation speaks at full volume; how loud each ends up being is learned, through the scale of W_r.

From nodes to a patient. After the last layer each visit has a vector. The visits of one patient are averaged into a single patient vector, and a small network turns that into the colour prediction. A second small network predicts each visit on its own, and the two losses are trained together.

One consequence worth knowing. Every neighbour weight α_{ij}^{r} is normalised to sum to one, which makes each layer an average. If all nodes start with the same input vector, they all stay identical forever, no matter how the graph is wired — the model cannot even count a node's neighbours. This is why the featureless ablation needs the node inputs to differ from each other before the graph can do anything at all.